

2주차_논문과제_23기_정준호

Attention Is All You Need

1. Introduction

1.1. 순환 모델의 지배와 한계

- RNN, LSTM, GRU 기반의 모델은 언어 모델링과 기계 번역에서 오랫동안 최고 수준의 성능을 유지해 왔습니다.
- 순환 모델은 입력과 출력 시퀀스의 위치를 따라 계산을 전개하며, 이전 은닉 상태 $h(t-1)$ 과 t 시점의 입력으로부터 $h(t)$ 를 생성합니다.
- 이러한 순차적 특성 때문에 학습 예제 내부에서의 병렬화가 원천적으로 불가능하며, 시퀀스가 길어질수록 메모리 제약으로 배치 처리도 어려워집니다.
- 인수분해 기법이나 조건부 연산으로 계산 효율은 개선되었지만, 순차 계산이라는 근본적인 제약은 그대로 남아 있습니다.

1.2. Attention의 도입과 논문의 제안

- Attention 메커니즘은 입출력 시퀀스 내의 거리와 무관하게 의존성을 모델링할 수 있어 다양한 과제에서 필수 요소로 자리 잡았습니다.
- 다만 대부분의 경우 attention은 순환 신경망과 함께 사용되는 보조적인 장치였습니다.
- 본 논문은 순환 구조를 완전히 배제하고 attention만으로 입력과 출력 사이의 전역 의존성을 모델링하는 Transformer를 제안합니다.
- Transformer는 훨씬 높은 수준의 병렬화를 가능하게 하며, P100 GPU 8장으로 12시간 학습하는 것만으로 새로운 최고 성능에 도달했습니다.

2. Background

2.1. 순차 연산을 줄이려는 기존 시도

- Extended Neural GPU, ByteNet, ConvS2S는 합성곱 신경망을 기본 블록으로 삼아 순차 연산을 줄이고자 했습니다.

- 이들 모델에서 임의의 두 위치 사이의 신호를 연결하는 데 필요한 연산 수는 거리에 따라 증가합니다. ConvS2S는 선형으로, ByteNet은 로그에 비례해 늘어납니다.
- 이 때문에 멀리 떨어진 위치 사이의 의존성을 학습하기가 더 어려워집니다.
- Transformer는 이 연산 수를 상수로 줄입니다. attention 가중 평균으로 인해 유효 해상도가 낮아지는 부작용은 Multi-Head Attention으로 상쇄합니다.

2.2. Self-Attention과 관련 연구

- Self-attention은 하나의 시퀀스 안에서 서로 다른 위치들을 연결해 그 시퀀스의 표현을 계산하는 방식으로, 독해와 요약, 문장 표현 학습 등에서 성공적으로 사용되었습니다.
- End-to-end memory network는 시퀀스 정렬 순환 대신 recurrent attention에 기반하며, 단순 언어 질의응답과 언어 모델링에서 좋은 성능을 보였습니다.
- 그러나 순환이나 합성곱 없이 self-attention만으로 입출력 표현을 계산하는 변환 모델은 Transformer가 처음입니다.

3. Model Architecture

3.1. Encoder and Decoder Stacks

- **전체 골격:** 대부분의 경쟁력 있는 신경망 시퀀스 변환 모델과 마찬가지로 encoder-decoder 구조를 따릅니다. 인코더가 입력 시퀀스를 연속 표현으로 사상하면, 디코더가 이를 받아 한 번에 한 토큰씩 출력을 생성합니다.
- **Encoder:** 동일한 층 6개를 쌓았으며, 각 층은 두 개의 하위 층으로 구성됩니다.
 - Multi-Head Self-Attention
 - Position-wise Fully Connected Feed-Forward Network
- **Decoder:** 마찬가지로 6개 층이며, 위의 두 하위 층에 인코더 출력을 대상으로 하는 Multi-Head Attention이 추가되어 총 세 개입니다.
- **Residual과 정규화:** 모든 하위 층에 잔차 연결을 적용한 뒤 층 정규화를 수행하므로, 각 하위 층의 출력은 $\text{LayerNorm}(x + \text{Sublayer}(x))$ 가 됩니다.
- 잔차 연결이 성립하려면 차원이 일치해야 하므로, 임베딩을 포함한 모든 하위 층의 출력 차원을 $d_{\text{model}} = 512$ 로 통일했습니다.

- **Masking:** 디코더의 self-attention에는 마스크를 적용해 특정 위치가 자기보다 뒤쪽 위치를 참조하지 못하게 합니다. 출력 임베딩을 한 칸 뒤로 미는 처리와 결합되어, i 번째 예측이 i 보다 앞선 위치의 정보에만 의존하도록 보장합니다.

3.2. Scaled Dot-Product Attention

- Attention은 query와 key-value 쌍의 집합을 출력으로 사상하는 함수로 기술할 수 있으며, 출력은 value들의 가중합입니다. 각 value의 가중치는 query와 대응하는 key의 호환성 함수로 계산됩니다.
- 계산식은 다음과 같습니다.

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$$

- **연산 순서:** $\text{MatMul}(QK^T) \rightarrow \text{Scale}(1/\sqrt{d_k}) \rightarrow \text{Mask}(\text{선택적}) \rightarrow \text{SoftMax} \rightarrow \text{MatMul}(V)$
- **Additive attention과의 비교:** 두 방식의 이론적 복잡도는 유사하지만, dot-product 방식은 고도로 최적화된 행렬 곱 코드로 구현할 수 있어 실제로는 훨씬 빠르고 메모리 효율적입니다.
- **스케일링이 필요한 이유:** d_k 가 작을 때는 두 방식의 성능이 비슷하지만, d_k 가 커지면 내적 값의 크기가 커져 softmax가 기울기가 극히 작은 영역으로 밀려납니다. 이를 막기 위해 내적 결과를 $\sqrt{d_k}$ 로 나눕니다.

3.3. Multi-Head Attention

- d_{model} 차원의 query, key, value로 단일 attention을 수행하는 대신, 서로 다른 학습된 선형 투영을 통해 d_k, d_k, d_v 차원으로 h 번 투영하는 방식이 더 유리하다는 것을 확인했습니다.
- 투영된 각 버전에 대해 attention을 병렬로 수행하고, 결과를 이어 붙인 뒤 다시 한 번 선형 투영해 최종 출력을 만듭니다.
- **하이퍼파라미터:** $h = 8, d_k = d_v = d_{\text{model}} / h = 64$
- 각 헤드의 차원을 축소했기 때문에 전체 연산 비용은 전체 차원을 사용하는 단일 헤드 attention과 유사합니다.
- **효과:** 단일 attention은 가중 평균 과정에서 여러 위치의 정보가 뭉개지지만, 여러 헤드를 사용하면 서로 다른 표현 부분공간의 정보를 서로 다른 위치에서 동시에 참조할 수 있습니다.

다.

3.4. 모델 내 Attention의 세 가지 활용

- **Encoder-Decoder Attention:** query는 이전 디코더 층에서, key와 value는 인코더의 최종 출력에서 가져옵니다. 디코더의 모든 위치가 입력 시퀀스의 모든 위치를 참조할 수 있으며, 기존 seq2seq 모델의 attention과 같은 역할을 합니다.
- **Encoder Self-Attention:** query, key, value가 모두 이전 인코더 층의 출력에서 나옵니다. 각 위치가 이전 층의 모든 위치를 참조합니다.
- **Decoder Self-Attention:** 자기 자신을 포함해 그 이전까지의 모든 위치를 참조합니다. 자기회귀 성질을 유지하기 위해 잘못된 방향의 연결에 해당하는 softmax 입력값을 음의 무한대로 설정해 차단합니다.

3.5. Position-wise Feed-Forward Networks

- 인코더와 디코더의 각 층에는 attention 하위 층과 별개로 완전 연결 피드포워드 네트워크가 포함되며, 각 위치에 독립적으로 그리고 동일하게 적용됩니다.
- 두 번의 선형 변환 사이에 ReLU 활성화가 들어가는 구조입니다.

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

- 층마다 파라미터는 다르며, 커널 크기가 1인 합성곱 두 개로도 동일하게 기술할 수 있습니다.
- 입출력 차원은 $d_{\text{model}} = 512$, 내부 층 차원은 $d_{\text{ff}} = 2048$ 입니다.

3.6. Embeddings and Softmax

- 입력 토큰과 출력 토큰을 d_{model} 차원 벡터로 변환하기 위해 학습된 임베딩을 사용하고, 디코더 출력을 다음 토큰 확률로 바꾸기 위해 학습된 선형 변환과 softmax를 사용합니다.
- 두 임베딩 층과 softmax 직전의 선형 변환은 동일한 가중치 행렬을 공유합니다.
- 임베딩 층에서는 해당 가중치에 $\sqrt{d_{\text{model}}}$ 을 곱합니다.

3.7. Positional Encoding

- 모델에 순환도 합성곱도 없으므로 시퀀스의 순서 정보를 별도로 주입해야 합니다. 이를 위해 인코더와 디코더 스택 하단의 입력 임베딩에 positional encoding을 더합니다.

- Positional encoding은 임베딩과 동일한 d_{model} 차원을 가지므로 단순 덧셈이 가능합니다.
- 주파수가 서로 다른 사인, 코사인 함수를 사용합니다.

$$\begin{aligned} \text{PE}(\text{pos}, 2i) &= \sin(\text{pos} / 10000^{(2i/d_{\text{model}})}) \\ \text{PE}(\text{pos}, 2i+1) &= \cos(\text{pos} / 10000^{(2i/d_{\text{model}})}) \end{aligned}$$

- 파장은 2π 에서 $10000 \cdot 2\pi$ 까지 기하급수적으로 늘어납니다.
- 임의의 고정된 offset k 에 대해 $\text{PE}(\text{pos}+k)$ 가 $\text{PE}(\text{pos})$ 의 선형 함수로 표현되기 때문에, 모델이 상대적인 위치를 쉽게 학습할 것이라는 가정에서 이 함수를 선택했습니다.
- 학습형 positional embedding과 비교 실험한 결과 성능은 거의 동일했으나, 학습 시 접한 것보다 긴 시퀀스로도 외삽이 가능할 것이라는 이유로 사인 함수 버전을 채택했습니다.

4. Why Self-Attention

4.1. 비교 기준

- **총당 총 연산 복잡도:** 한 층을 계산하는 데 드는 전체 비용입니다.
- **병렬화 가능한 연산량:** 필요한 최소 순차 연산 수로 측정합니다.
- **장거리 의존성 경로 길이:** 네트워크 내에서 임의의 두 위치를 잇는 전방 및 후방 신호의 최대 경로 길이로, 짧을수록 장거리 의존성 학습이 쉬워집니다.

4.2. 계층별 비교 결과

- **Self-Attention:** 층당 $O(n^2 \cdot d)$, 순차 연산 $O(1)$, 최대 경로 길이 $O(1)$
- **Recurrent:** 층당 $O(n \cdot d^2)$, 순차 연산 $O(n)$, 최대 경로 길이 $O(n)$
- **Convolutional:** 층당 $O(k \cdot n \cdot d^2)$, 순차 연산 $O(1)$, 최대 경로 길이 $O(\log_k n)$
- **Restricted Self-Attention:** 층당 $O(r \cdot n \cdot d)$, 순차 연산 $O(1)$, 최대 경로 길이 $O(n/r)$
- 기계 번역에서 주로 사용되는 word-piece나 byte-pair 표현처럼 시퀀스 길이 n 이 표현 차원 d 보다 작은 경우, self-attention 층이 순환 층보다 연산량 면에서도 유리합니다.
- 매우 긴 시퀀스에 대응하기 위해서는 각 출력 위치가 주변 r 개 위치만 참조하는 제한된 self-attention을 고려할 수 있으나, 이 논문에서 실험하지는 않았습니다.

- 합성곱 층은 커널 폭 k 가 n 보다 작으면 모든 위치 쌍을 연결하지 못해 층을 여러 개 쌓아야 하며, 분리형 합성곱을 쓰더라도 복잡도는 self-attention과 FFN을 결합한 수준과 비슷합니다.
- **부수적 이점:** attention 분포를 관찰하면 개별 헤드가 어떤 역할을 학습했는지 확인할 수 있어, 모델의 해석 가능성 측면에서도 이득이 있습니다.

5. Training

5.1. 학습 데이터 및 배치

- **WMT 2014 English-German:** 약 450만 문장 쌍. 약 37000개의 토큰으로 구성된 공유 source-target 어휘를 byte-pair encoding으로 구축했습니다.
- **WMT 2014 English-French:** 약 3600만 문장. 32000개의 word-piece 어휘를 사용했습니다.
- 문장 쌍은 대략적인 시퀀스 길이 기준으로 묶었으며, 각 학습 배치는 약 25000개의 source 토큰과 25000개의 target 토큰을 포함합니다.

5.2. 하드웨어 및 학습 시간

- NVIDIA P100 GPU 8장으로 학습했습니다.
- **Base 모델:** 스텝당 약 0.4초, 총 100,000 스텝을 12시간 동안 학습했습니다.
- **Big 모델:** 스텝당 약 1.0초, 총 300,000 스텝을 3.5일 동안 학습했습니다.

5.3. Optimizer

- Adam을 사용했으며 $\beta_1 = 0.9$, $\beta_2 = 0.98$, $\epsilon = 10^{-9}$ 입니다.
- 학습률은 아래 수식에 따라 변화시켰습니다.

```
lr_rate = d_model(-0.5) · min(step(-0.5), step · warmup_steps(-1.5))
```

- warmup_steps = 4000까지는 학습률을 선형으로 증가시키고, 이후에는 스텝 수의 역제곱근에 비례해 감소시킵니다.

5.4. 정규화

- **Residual Dropout:** 각 하위 층의 출력이 잔차와 더해지고 정규화되기 전에 dropout을 적용하며, 임베딩과 positional encoding의 합에도 적용합니다. base 모델은 $P_{drop} = 0.1$ 을 사용했습니다.
- **Label Smoothing:** $\epsilon_{ls} = 0.1$ 을 적용했습니다. 모델이 더 불확실해지도록 학습되므로 perplexity는 나빠지지만 정확도와 BLEU는 개선됩니다.

6. Results

6.1. Machine Translation

- **English-German:** Transformer (big) 모델이 28.4 BLEU를 기록하며, 앙상블을 포함한 기존 최고 성능을 2.0 BLEU 이상 앞섰습니다.
- Base 모델 역시 기존에 발표된 모든 단일 모델과 앙상블을 능가했으며, 학습 비용은 경쟁 모델의 일부에 불과했습니다.
- **English-French:** Transformer (big) 모델이 41.0 BLEU를 달성하여 기존에 발표된 모든 단일 모델을 앞섰고, 학습 비용은 직전 최고 모델의 1/4 미만이었습니다. 이 모델은 dropout을 0.3 대신 0.1로 두었습니다.
- **추론 설정:** beam size 4, length penalty $\alpha = 0.6$ 을 사용했습니다. 출력 최대 길이는 입력 길이 + 50으로 두되 가능한 경우 조기에 종료했습니다.
- **체크포인트 평균:** base 모델은 10분 간격으로 저장한 마지막 5개, big 모델은 마지막 20개 체크포인트를 평균해 사용했습니다.

6.2. Model Variations

- newstest2013 개발셋에서 base 모델의 구성 요소를 바꿔가며 성능 변화를 측정했습니다.
- **(A) 헤드 수:** 연산량을 고정한 채 헤드 수와 d_k , d_v 를 변화시킨 결과, 단일 헤드는 최적 설정보다 0.9 BLEU 낮았고, 헤드가 지나치게 많아도 품질이 떨어졌습니다.
- **(B) d_k :** attention key 차원을 줄이면 모델 품질이 저하되었습니다. 호환성을 판단하는 일이 쉽지 않으며, 단순 내적보다 정교한 호환성 함수가 유리할 수 있음을 시사합니다.
- **(C), (D) 모델 크기와 dropout:** 모델이 클수록 성능이 좋았고, dropout이 과적합 방지에 도움이 되었습니다.

- **(E) Positional Encoding:** 사인 함수 방식을 학습형 positional embedding으로 교체한 결과, base 모델과 거의 동일한 성능을 보였습니다.

6.3. English Constituency Parsing

- Transformer가 다른 과제로도 일반화되는지 확인하기 위해 영어 구문 분석 실험을 수행했습니다. 출력이 강한 구조적 제약을 받고 입력보다 훨씬 길다는 점에서 까다로운 과제입니다.
 - WSJ Penn Treebank의 약 4만 문장으로 학습한 4층 모델($d_{\text{model}} = 1024$)을 사용했으며, 준지도 설정에서는 약 1700만 문장 규모의 코퍼스를 추가로 활용했습니다.
 - 과제별 튜닝을 거의 하지 않았음에도 WSJ만 사용한 설정에서 91.3 F1, 준지도 설정에서 92.7 F1을 기록했습니다.
 - Recurrent Neural Network Grammar를 제외한 기존의 모든 모델보다 좋은 결과입니다.
-

7. Conclusion

- 전적으로 attention에 기반한 최초의 시퀀스 변환 모델을 제시했으며, encoder-decoder 구조에서 가장 흔히 쓰이던 순환 층을 Multi-Head Self-Attention으로 대체했습니다.
 - 번역 과제에서 순환이나 합성곱 기반 구조보다 훨씬 빠르게 학습되며, WMT 2014 English-German과 English-French 모두에서 새로운 최고 성능을 달성했습니다.
 - 저자들은 향후 텍스트 이외의 입출력 양식으로 Transformer를 확장하고, 이미지나 오디오, 비디오와 같은 대규모 입출력을 효율적으로 다루기 위한 제한된 attention 메커니즘을 연구할 계획이라고 밝혔습니다.
 - 생성 과정을 덜 순차적으로 만드는 것 또한 연구 목표로 제시했습니다.
-